

✓ ZoyaSofts Internship Training

Week 1 — AI & Python Foundations

Day 5 — AI Problem Solving

4×4 Grid Maze

We will model the following maze as a state space:

S # # # .

. . G

Symbols

- S = Start state
- G = Goal state
- . = Open state

• = Blocked state

State Space

Each open cell in the maze represents a possible state.

Moving from one open cell to another represents a transition between states.

The objective is to find a path from the Start state (S) to the Goal state (G).

BFS vs DFS

Breadth-First Search (BFS)

BFS explores the state space level by level.

It checks all nearby states before moving to states that are farther away.

For an unweighted maze, BFS can find the shortest path from the start to the goal.

Depth-First Search (DFS)

DFS explores one path as deeply as possible before backtracking and trying another path.

DFS can find a path to the goal, but it does not guarantee the shortest path in an unweighted maze.

Comparison

Algorithm	How it explores	Shortest path?
BFS	Level by level	Yes, for an unweighted maze
DFS	Depth first, then backtracks	Not guaranteed

Result for This Maze

For our 4×4 maze, BFS is the appropriate algorithm when we want the shortest path.

DFS may also reach the goal, but it can explore a longer route first.

A* and Minimax

A* Search

A* is a guided search algorithm.

It uses a heuristic to estimate how close a state is to the goal.

This can make searching more efficient than exploring every possible path.

A* is commonly useful for pathfinding problems.

Minimax

Minimax is a decision-making concept used in competitive or game-playing situations.

It evaluates possible moves by considering the choices of an opponent.

Key Differences

- BFS → Explores level by level.
- DFS → Explores deeply before backtracking.
- A* → Uses a heuristic to guide the search.
- Minimax → Used for decision-making in competitive situations.

BFS and DFS Maze Exploration

Maze

S # # . . . # .

.. G

BFS Exploration

BFS explores the maze level by level.

Starting from S, BFS first explores the nearby cells and then continues to the next levels.

One possible shortest path is:

$S \rightarrow (1,2) \rightarrow (1,3) \rightarrow (1,4) \rightarrow (2,4) \rightarrow (3,4) \rightarrow (4,4) \rightarrow G$

BFS finds this shortest path because it explores states according to their distance from the starting state.

DFS Exploration

DFS explores one available path as deeply as possible before backtracking.

Depending on the order in which neighboring cells are explored, DFS may take a different route before reaching the goal.

DFS does not guarantee the shortest path in an unweighted maze.

Conclusion

For this maze, BFS is preferred when the objective is to find the shortest path.

DFS can be useful when we only need to find a path and do not require the shortest path.

✓ Day 5 — Conclusion

Today I learned how AI problems can be represented as a state space and solved using search algorithms.

What I Learned

- A state represents a possible position or situation.
- A state space contains the possible states of a problem.
- BFS explores level by level and can find the shortest path in an unweighted maze.
- DFS explores deeply before backtracking and does not guarantee the shortest path.
- A* uses a heuristic to guide the search toward the goal.
- Minimax is used for decision-making in competitive or game-playing situations.

Learning Outcome

I can describe a problem as a state space and explain the basic differences between BFS, DFS, A*, and Minimax.

Start coding or [generate](#) with AI.